Computer Programming 1

Laboratory - recursive functions (part 1)

(20-10-2026)

Example 1a: factorial – *iterative* version

[optional] If you haven't implemented it yet

- Write a program that calculates the factorial of a positive integer* received by the user

- Constraints:
  - Implements the iterative version of the function to compute the factorial.
  - Do not use arrays or strings, or any other complex data type.

$$n! := \prod_{k=1}^{n} k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

n	*n*!
0	1
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
…	…

* condition to verify

Example 1b: factorial - recursive

- Write a program that calculates the factorial of a positive integer* received by the user

- Constraint: Use a recursive function to compute the factorial.
- Constraint: Do not use arrays or strings, or any other complex data type.

$$n! := \prod_{k=1}^{n} k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

n	*n*!
0	1
1	1
2	2
3	6
4	24
5	120
6	720
7	5040
8	40320
…	…

* condition to verify

Example 1c: factorial and sum

- Write a program that calculates the factorial of a positive integer* received by the user
- Constraint: Use a recursive function to compute the factorial.
- Do not use arrays or strings, or any other complex data type.

$$n! := \prod_{k=1}^{n} k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

n	*n*!	Sum
0	1	0
1	1	1
2	2	3
3	6	6
4	24	10
5	120	15
6	720	21
7	5040	28
8	40320	36
…	…	…

- At the same time, the program has to compute also the sum of the integer used to compute the factorial of the provided positive integer (i.e., named ***summation*** of the numbers)

Implementation constraint: do not use a recursive function to compute the summation

* condition to verify

Example 1d: factorial and sum

- Write a program that calculates the factorial of a positive integer* received by the user
- Constraint: Use a recursive function to compute the factorial.
- Do not use arrays or strings, or any other complex data type.

$$n! := \prod_{k=1}^{n} k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

- At the same time, the program has to compute also the sum of the integer used to compute the factorial of the provided positive integer (i.e., named ***summation*** of the numbers)

n	*n*!	Sum
0	1	0
1	1	1
2	2	3
3	6	6
4	24	10
5	120	15
6	720	21
7	5040	28
8	40320	36
…	…	…

Implementation constraint: use a recursive function also to compute the summation

* condition to verify

Example 2: Division

- Write a program that calculates the modulo operation (‘%’ or ‘mod’) between two integer numbers, by using a recursive function and not the ‘%’ operator.

- The user must insert both dividend and divisor (i.e., two integer and positive numbers*), and return the quotient and the rest
- The ‘%’ operator can be used to verify the result obtained by using the recursive function

E.g., 12 mod 6 or 12 % 6

- Dividend: 12
- Divisor:6
- Result = 2
- Reminder = 0

E.g., 13 mod 6 or 13%6

- Dividend: 13
- Divisor:6
- Result = 2
- Reminder = 1

* condition to verify

Example 3: Print binary

- Write a program that takes a number* as input and prints the binary representation on the screen using a recursive procedure.

- Do not use any assumption on the number of digits to be used for the binary representation
- Do not use any libraries (apart iostream).
- Do not use arrays or strings, or any other complex data type.

```
E.g.,
    13 => 1101
    345 => 101011001
    12045 => 10111100001101
```

[optional] Example 3b: Print binary - iterative

If you have not implemented it before, you can also implement the iterative version

* condition to verify

Example 4: Palindrome

Write a program that takes a positive integer number* as input and checks if it is a palindrome number.

Constraints:

- Implement the recursive function by using a **helper** function and a **wrapper** function.
- Do not use arrays or strings, or any other complex data type.

Remarks

- "A palindromic number is a number that remains the same when its digits are reversed."
- E.g., palindrome numbers: 16461, 121, 13531, 3223

* condition to verify

Example 5: Sum of digits

- Write a recursive function that, given a positive integer number* as input, returns the sum of the digits that make up the integer.

  - Do not use any assumption on the number of digits used
  - Do not use any libraries (apart iostream).
  - Do not use arrays or strings, or any other complex data type

```
E.g.,
25 => 7
4526 => 17
```

[optional] Example 5b: Sum of digits - iterative

If you have not implemented it before, you can also implement the iterative version

* condition to verify

Example 6: Print character

- Write a program that takes two characters as input and prints on the screen all the characters included through a recursive procedure.

```
E.g.,
(a, g)=> «a,b,c,d,e,f,g»
```

Example 7a: Compute PI with Leibniz - iterative

- Compute the Leibniz PI number as:

$$1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots = \frac{\pi}{4},$$

- Constraints:
  - Do not use the recursion, but rather implement it iteratively
  - Do not use any assumption on the number of numbers considered
  - Do not use any libraries (apart iostream).
  - Do not use arrays or strings, or any other complex data type

Example 7b: Compute PI with Leibniz - recursive

- Compute the Leibniz PI number as :

$$1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots = \frac{\pi}{4},$$

- Constraints:
  - Use the recursion function for the computation
  - Do not use any assumption on the number of numbers considered
  - Do not use any libraries (apart iostream).
  - Do not use arrays or strings, or any other complex data type

Example 7c: Compute PI with Leibniz - recursive

- Compute the Leibniz PI number as :

$$1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \cdots = \frac{\pi}{4},$$

- Constraints:
  - Use the recursion function for the computation
  - Do not use any assumption on the number of numbers considered
  - Do not use any libraries (apart iostream).
  - Do not use arrays or strings, or any other complex data type

[optional] Implements a solution based on a tail-recursive function. If your first solution is already tail-recursive, try to implement a different recursive solution that is not tail-recursive.